

Catalyst.jl Integration

EdgeBasedModels.jl Vignette 04

Simon Frost

2026-05-14

Table of contents

Introduction	1
Setup	1
SIR via Catalyst	2
Building and solving the model	2
Comparison with <code>build_sir</code>	5
SEIR via Catalyst	7
Custom multi-stage model via Catalyst	11
Symbolic R_0	14
Simulation validation	15
Benefits of the Catalyst adapter	16

Introduction

For users already familiar with [Catalyst.jl](#)'s reaction network DSL, `EdgeBasedModels.jl` provides a convenient bridge: the `progression_from_catalyst` function converts a Catalyst `ReactionSystem` into a `DiseaseProgression` that can be plugged directly into any edge-based configuration model. This lets you leverage Catalyst's expressive chemical kinetics notation for specifying disease compartment transitions, while `EdgeBasedModels` handles the network epidemic equations automatically.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using OrdinaryDiffEq
using Plots
import Catalyst
```

We use `import Catalyst` rather than `using Catalyst` to avoid namespace collisions with `ModelingToolkit` (both packages export symbols like `@parameters`).

SIR via Catalyst

In the standard edge-based SIR model, the disease progression is simply $I \xrightarrow{\gamma} R$. We can express this as a Catalyst reaction network:

```
@parameters β γ γ_cat
rn = Catalyst.@reaction_network begin
    γ_cat, I --> R
end; nothing
```

Now we convert this `ReactionSystem` into a `DiseaseProgression`. The `transmission_rates` dictionary specifies which compartments can transmit infection across an edge, and at what rate:

```
progression = progression_from_catalyst(
    rn;
    transmission_rates = Dict{:I => β, :R => 0},
    entry = :I
)
```

```
DiseaseProgression(:S, :I, DiseaseStage[DiseaseStage(:I, β), DiseaseStage(:R, 0)], DiseaseTr
```

The `entry = :I` argument tells the model that newly infected nodes enter the *I* compartment (as opposed to an exposed class).

Building and solving the model

We pair the progression with a Poisson degree distribution to build a `StaticConfigurationModel`:

```

@parameters κ
pgf = poisson_pgf(κ)
scm = StaticConfigurationModel(pgf, progression)
result = build_edge_system(scm)

```

EdgeModelSystem(Model edge_based_model:

Equations (5):

5 standard: see equations(edge_based_model)

Unknowns (5): see unknowns(edge_based_model)

pop_R(t)

pop_I(t)

phi_R(t)

phi_I(t)

⌈

Parameters (4): see parameters(edge_based_model)

κ

ρ

β

γ_cat

Observed (3): see observed(edge_based_model), Dict{Symbol, Any}(:R => pop_R(t), :φ_I => phi_I

Set up numeric parameters and solve:

i Note

$R_0 = 2$ anchor. For the Poisson($\kappa = 10$) SIR example, $T = 2/10$ and with $\gamma = 0.25$ the comparable per-edge rate is $\beta = 0.0625$. We seed 1% of the population.

```

κ_val = 10.0
γ_val = 0.25
R0_target = 2.0
T_val = R0_target / κ_val
β_val = T_val * γ_val / (1 - T_val)
ε = 0.001
tspan = (0.0, 40.0)

ic = default_initial_conditions(result; ε = ε)
params = Dict(κ => κ_val, β => β_val, γ_cat => γ_val)
prob = ODEProblem(result.system, merge(ic, params), tspan)
sol = solve(prob, Tsit5())

```

```

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 17-element Vector{Float64}:
 0.0
 0.13910135194928805
 0.7934475769879635
 1.860389932156775
 3.116396208533559
 4.675956941087338
 6.474177973429059
 8.539035769590395
10.86502252940332
13.508843513796641
16.6325527366216
20.117814388345924
23.897317691439458
27.93889670476307
32.50146326920048
37.24099758319997
40.0

```

```

u: 17-element Vector{Vector{Float64}}:
 [0.0, 0.001, 0.0, 0.0010000000000000009, 1.0]
 [3.5692254628371004e-5, 0.0010530665393326554, 3.554063551525515e-5, 0.0010443329995669583,
 [0.00023007092154663704, 0.0013318347914273027, 0.00022505057022450612, 0.0012805925001933,
 [0.0006589459062908662, 0.0019137399900481442, 0.000630200239298163, 0.0017849355972163077,
 [0.0014019537446062027, 0.0028786016216350127, 0.0013156971140657434, 0.002635933973659038,
 [0.0028534387156879705, 0.0047091782721589565, 0.002636343103371392, 0.004267188108632689,
 [0.0056866851393438155, 0.00820551643881826, 0.005195639069916711, 0.007397652740766191, 0.
 [0.011572047291637773, 0.015293030334830815, 0.01049280663697351, 0.013749069330251707, 0.9
 [0.024297868870993236, 0.029967971849563003, 0.021917246519485917, 0.026869282571198853, 0.
 [0.05317413134365906, 0.06021982723941671, 0.047741275802411574, 0.05371736383006132, 0.988
 [0.12095212107927708, 0.11610596879146957, 0.10782823444499595, 0.1022727968145017, 0.97304
 [0.24957450427379946, 0.17347860776486215, 0.21959981389699162, 0.14855334466742204, 0.9451
 [0.42016970522000197, 0.1759947140571007, 0.36228528316948416, 0.14330781531524736, 0.90942
 [0.5753670738893534, 0.12727521472911588, 0.48472732733809765, 0.09673312944584704, 0.87881
 [0.6866791828975831, 0.07081467639081336, 0.5662882785729862, 0.04963351107216348, 0.858427
 [0.7467123370184704, 0.03422098074719225, 0.6069522759917183, 0.02224297277601459, 0.848261
 [0.765720953017894, 0.021759511269806863, 0.619089248849319, 0.013618903226051936, 0.845227

```

```

κ_val_num = 10.0
ψ(x) = exp(κ_val_num * (x - 1))
θ_vals = compartment(sol, result, :θ)

```

```

S_vals = compartment(sol, result, :S)
R_vals = compartment(sol, result, :R)
I_vals = compartment(sol, result, :I)

plot(sol.t, S_vals, label="S", lw=2)
plot!(sol.t, I_vals, label="I", lw=2)
plot!(sol.t, R_vals, label="R", lw=2)
xlabel!("Time")
ylabel!("Fraction")
title!("SIR (Catalyst-defined progression)")

```

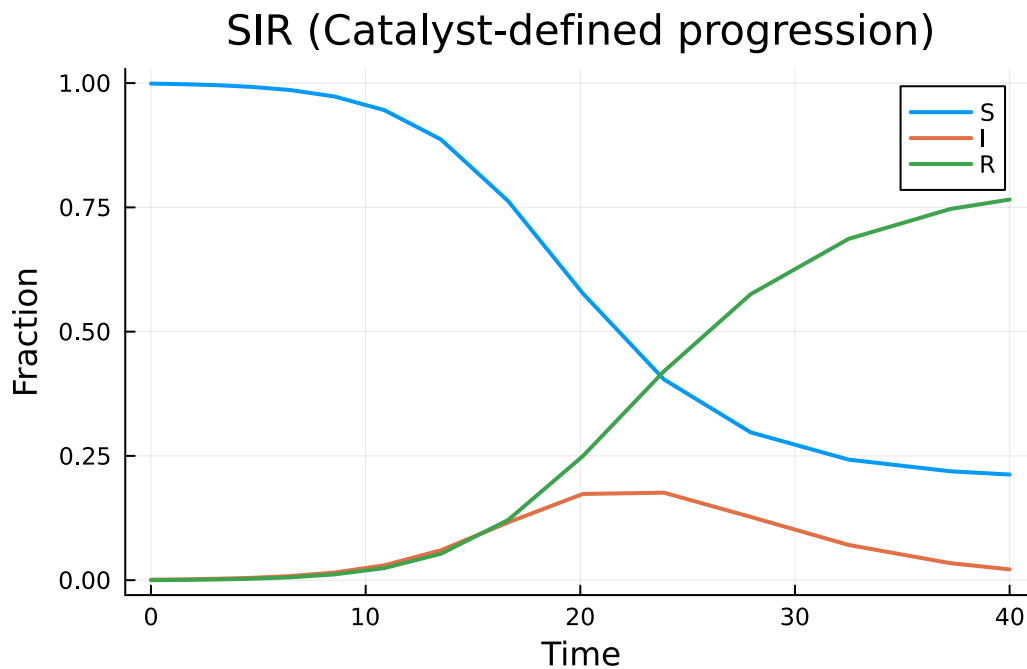


Figure 1: SIR epidemic on a Poisson network, defined via Catalyst.jl.

Comparison with build_sir

To verify that the Catalyst pathway produces identical results, we can compare against the built-in build_sir:

```

result_builtin = build_sir(pgf, β, γ)
ic_builtin = default_initial_conditions(result_builtin; ε = ε)
params_builtin = Dict(κ ⇒ κ_val, β ⇒ β_val, γ ⇒ γ_val)

```

```

prob_builtin = ODEProblem(result_builtin.system, merge(ic_builtin, params_builtin), tspan)
sol_builtin = solve(prob_builtin, Tsit5())

```

```

θ_builtin = compartment(sol_builtin, result_builtin, :θ)
S_builtin = ψ.(θ_builtin)

```

17-element Vector{Float64}:

```

1.0
0.9999111523583968
0.9994375318190064
0.9984257398512925
0.9967161608476413
0.9934308143059295
0.9870948965911975
0.9741090539243975
0.9466809805606917
0.8874942917306587
0.7637073704854087
0.5775273174608195
0.40425346548889285
0.2976555545096365
0.2427506008988106
0.21928551211246103
0.21273178712560806

```

```

R_builtin = compartment(sol_builtin, result_builtin, :R)
I_builtin = 1.0 .- S_builtin .- R_builtin

```

```

plot(sol.t, S_vals, label="S (Catalyst)", lw=2, color=1)
plot!(sol_builtin.t, S_builtin, label="S (built-in)", lw=2, ls=:dash, color=1)
plot!(sol.t, I_vals, label="I (Catalyst)", lw=2, color=2)
plot!(sol_builtin.t, I_builtin, label="I (built-in)", lw=2, ls=:dash, color=2)
plot!(sol.t, R_vals, label="R (Catalyst)", lw=2, color=3)
plot!(sol_builtin.t, R_builtin, label="R (built-in)", lw=2, ls=:dash, color=3)
xlabel!("Time")
ylabel!("Fraction")
title!("Catalyst vs built-in SIR")

```

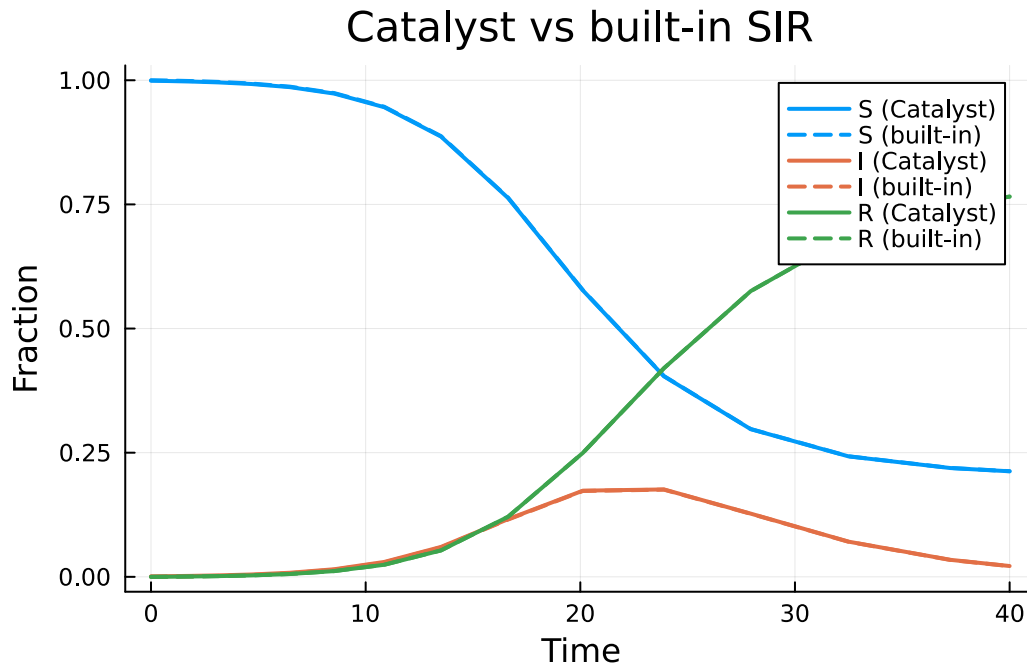
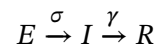


Figure 2: Comparison of Catalyst-defined SIR (solid) and built-in `build_sir` (dashed). The curves overlap exactly.

The solid and dashed lines overlap perfectly, confirming that both approaches produce the same ODE system.

SEIR via Catalyst

The SEIR model adds a latent (exposed) period before infectiousness. The compartment transitions are:



In Catalyst notation:

```
@parameters σ_cat
rn_seir = Catalyst.@reaction_network begin
    σ_cat, E --> I
    γ_cat, I --> R
end; nothing
```

Converting to a `DiseaseProgression`, we specify that the E and R compartments do not transmit ($\beta = 0$), only I does:

```
progression_seir = progression_from_catalyst(
    rn_seir;
    transmission_rates = Dict{:E => 0, :I =>  $\beta$ , :R => 0},
    entry = :E
)
```

```
DiseaseProgression(:S, :E, DiseaseStage[DiseaseStage(:E, 0), DiseaseStage(:I,  $\beta$ ), DiseaseSta
```

```
scm_seir = StaticConfigurationModel(pgf, progression_seir)
result_seir = build_edge_system(scm_seir)
```

```
EdgeModelSystem(Model edge_based_model:
```

```
Equations (7):
```

```
7 standard: see equations(edge_based_model)
```

```
Unknowns (7): see unknowns(edge_based_model)
```

```
pop_R(t)
```

```
pop_I(t)
```

```
pop_E(t)
```

```
phi_R(t)
```

```
⌘
```

```
Parameters (5): see parameters(edge_based_model)
```

```
 $\kappa$ 
```

```
 $\rho$ 
```

```
 $\beta$ 
```

```
 $\sigma_{cat}$ 
```

```
⌘
```

```
Observed (3): see observed(edge_based_model), Dict{Symbol, Any}(:R => pop_R(t), : $\phi_I$  => phi_I
```

Let's inspect the system. The SEIR edge-based model has 7 differential equations: θ , ϕ_E , ϕ_I , ϕ_R , plus per-stage population trackers (pop_E , pop_I , pop_R), plus algebraic observables for ϕ_S , S , E , I , and R :

```
println("Number of equations: ", length(ModelingToolkit.equations(result_seir.system)))
println("\nVariables: ", keys(result_seir.variables))
println("\nObservables: ", keys(result_seir.observables))
```


Number of equations: 7

Variables: [:R, : φ_I , :pop_I, :pop_R, : φ_R , : θ , : φ_E , :pop_E]

Observables: [:I, : φ_S , :S, :edge_hazard, :excess_hazard]

Solve and plot:

```
 $\sigma_{\text{val}} = 0.2$ 

ic_seir = default_initial_conditions(result_seir;  $\epsilon = \epsilon$ )
params_seir = Dict( $\kappa \Rightarrow \kappa_{\text{val}}$ ,  $\beta \Rightarrow \beta_{\text{val}}$ ,  $\sigma_{\text{cat}} \Rightarrow \sigma_{\text{val}}$ ,  $\gamma_{\text{cat}} \Rightarrow \gamma_{\text{val}}$ )
prob_seir = ODEProblem(result_seir.system, merge(ic_seir, params_seir), tspan)
sol_seir = solve(prob_seir, Tsit5())
```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 17-element Vector{Float64}:

```
0.0
0.152936027064522
0.64349338690379
1.3445914715284575
2.1988694944572162
3.282792112176137
4.591867282721296
6.199174244359496
8.157875770719697
10.59484997079258
13.68879177786621
17.804388625756822
23.255782187365377
28.627595568107367
34.31829740724162
39.93596075140407
40.0
```

u: 17-element Vector{Vector{Float64}}:

```
[0.0, 0.0, 0.001, 0.0, 0.0, 0.0010000000000000009, 1.0]
[5.716320606785936e-7, 2.9567009987684497e-5, 0.0009712844878947393, 5.698222049205534e-7,
[9.444785676853656e-6, 0.00011230840177256893, 0.0009015273450150443, 9.321643157782865e-6,
[3.770372187179559e-5, 0.00020615971627962914, 0.000847817125096481, 3.67106189322532e-5, 0.
[9.162025601818047e-5, 0.00029516256790721465, 0.0008325924185955709, 8.784758112662653e-5,
[0.00018425378825639175, 0.00038558722705535456, 0.0008636444016635161, 0.0001736054034034
```

```
[0.00032612164740247637, 0.0004799188056846442, 0.0009473233073355706, 0.00030176094838866,
[0.000541356908774147, 0.0005917859642169183, 0.0010947429440008442, 0.000491948366476419,
[0.0008664517728570345, 0.0007391094405623342, 0.0013262648562785881, 0.000774252783643664,
[0.0013807618094231484, 0.0009569680819426959, 0.0016937319958001884, 0.001215643920627732,
[0.0022525504474967552, 0.0013148899308097362, 0.002312819806521039, 0.0019588466534646927,
[0.0039311151677741555, 0.001993575688233569, 0.0034945794360946984, 0.003385364622764418,
[0.007545330900608262, 0.0034379319654517103, 0.006003767576798166, 0.006452984252243756, 0.
[0.013634279655796158, 0.005832890284832118, 0.010129793822469544, 0.011617311539703974, 0.
[0.024685714117534546, 0.01006286993745493, 0.01730194543698106, 0.020981413151688773, 0.00
[0.04320029230676159, 0.016815213177834944, 0.028434774032099774, 0.03664341259326816, 0.01
[0.043470262232602676, 0.01691042105391133, 0.02858939370037411, 0.036871571590099725, 0.01
```

```
θ_seir = compartment(sol_seir, result_seir, :θ)
S_seir = ψ.(θ_seir)
R_seir = compartment(sol_seir, result_seir, :R)
I_seir = 1.0 .- S_seir .- R_seir

plot(sol_seir.t, S_seir, label="S", lw=2)
plot!(sol_seir.t, I_seir, label="E + I", lw=2)
plot!(sol_seir.t, R_seir, label="R", lw=2)
xlabel!("Time")
ylabel!("Fraction")
title!("SEIR (Catalyst-defined progression)")
```

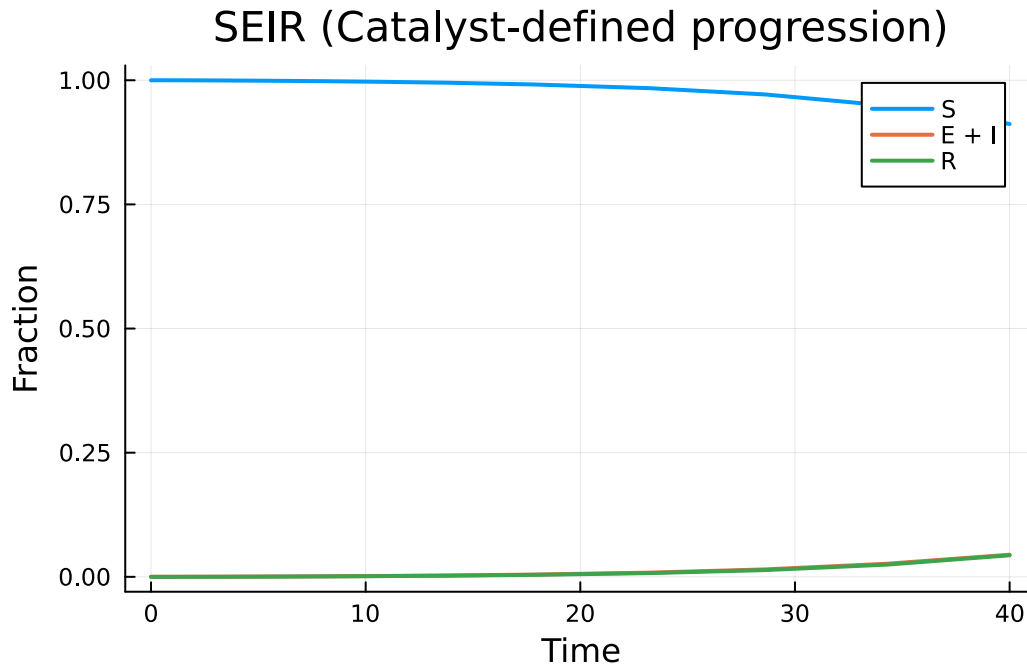
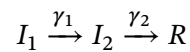


Figure 3: SEIR epidemic on a Poisson network, defined via Catalyst.jl. The exposed class delays the onset of infectiousness.

Custom multi-stage model via Catalyst

Catalyst's flexibility really shines when defining non-standard disease progressions. Consider a model with two sequential infectious stages, I_1 and I_2 , each with different transmission rates:



This could represent an initial acute phase (I_1 , high transmission) followed by a chronic phase (I_2 , lower transmission).

```
@parameters β1 β2 γ1_cat γ2_cat
rn_multi = Catalyst.@reaction_network begin
    γ1_cat, I1 --> I2
    γ2_cat, I2 --> R
end; nothing
```

```

progression_multi = progression_from_catalyst(
    rn_multi;
    transmission_rates = Dict(:I1  $\Rightarrow \beta_1$ , :I2  $\Rightarrow \beta_2$ , :R  $\Rightarrow 0$ ),
    entry = :I1
)

```

DiseaseProgression(:S, :I1, DiseaseStage[DiseaseStage(:I1, β_1), DiseaseStage(:I2, β_2), Disea

```

scm_multi = StaticConfigurationModel(pgf, progression_multi)
result_multi = build_edge_system(scm_multi)
println("Number of equations: ", length(ModelingToolkit.equations(result_multi.system)))
println("Variables: ", keys(result_multi.variables))

```

Number of equations: 7

Variables: [:R, :pop_I2, : φ _I2, :pop_I1, :pop_R, : φ _R, : θ , : φ _I1]

Solve with the acute phase having higher transmissibility:

```

 $\beta_1$ _val = 0.5 # high transmission in acute phase
 $\beta_2$ _val = 0.1 # lower transmission in chronic phase
 $\gamma_1$ _val = 0.3 # fast progression out of acute phase
 $\gamma_2$ _val = 0.05 # slow recovery from chronic phase

ic_multi = default_initial_conditions(result_multi;  $\epsilon = \epsilon$ )
params_multi = Dict(
     $\kappa \Rightarrow \kappa$ _val,
     $\beta_1 \Rightarrow \beta_1$ _val,  $\beta_2 \Rightarrow \beta_2$ _val,
     $\gamma_1$ _cat  $\Rightarrow \gamma_1$ _val,  $\gamma_2$ _cat  $\Rightarrow \gamma_2$ _val
)
prob_multi = ODEProblem(result_multi.system, merge(ic_multi, params_multi), tspan)
sol_multi = solve(prob_multi, Tsit5())

```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 36-element Vector{Float64}:

```

0.0
0.07221505633112481
0.17164998934568393
0.2894760913432517
0.42973616294583195

```

```

0.5886071437873984
0.7657551786996984
0.9598435414033647
1.1730462397620993
1.41456977153802
[?]
17.96376771398558
20.06425903639986
22.433762726255402
24.98372300804928
27.741914416815494
30.73313781123839
34.03664524041226
37.74965102100452
40.0
u: 36-element Vector{Vector{Float64}}:
 [0.0, 0.0, 0.001, 0.0, 0.0, 0.0010000000000000009, 1.0]
 [4.384036234781953e-8, 2.569703677360923e-5, 0.001396446140948816, 4.323341524212565e-8, 2.
 [2.9209384439267297e-7, 7.802869688119589e-5, 0.0021838268432146316, 2.8303722574497677e-7
 [1.0224502037953682e-6, 0.000178508480883014, 0.0036651312947435168, 9.727190625842816e-7,
 [2.937739345983008e-6, 0.0003887413961433046, 0.006717022949948937, 2.74357772194001e-6, 0.
 [7.624811080374049e-6, 0.0008419793413386871, 0.0132098293901211, 7.001808038307339e-6, 0.0
 [1.9087567232077527e-5, 0.0018723635756044206, 0.027725445731644573, 1.72795625866404e-5, 0
 [4.753668957393992e-5, 0.004306830648894955, 0.06095177654396458, 4.25506945679915e-5, 0.00
 [0.0001210182738032285, 0.010271615123501023, 0.1366325764873295, 0.00010735012997203087, 0
 [0.0003239868925266475, 0.025232910804333642, 0.2952053522264803, 0.00028479060487655026, 0
 [?]
 [0.4707952263209215, 0.5213780455974942, 0.007602349192436457, 0.11163933598530082, 0.03999
 [0.5229742478953786, 0.47275688439875146, 0.00405891389419033, 0.11524252760982001, 0.02919
 [0.5759248766263234, 0.4218745269289338, 0.002001555939852376, 0.11815332518391462, 0.02040
 [0.6264989083443238, 0.3723730458639724, 0.0009367298671138012, 0.1203215595396475, 0.01396
 [0.6745084088827552, 0.32489255929204824, 0.00041312687035704825, 0.1218986036777647, 0.009
 [0.7196606141857129, 0.2799863780052826, 0.0001708194364324247, 0.12301146332881234, 0.0058
 [0.7622997263596654, 0.23745563110275558, 6.497150046554699e-5, 0.12377959587770762, 0.0035
 [0.8025377924126014, 0.1972618880486186, 2.2303700002091224e-5, 0.12429118560395636, 0.0020
 [0.8235307220282455, 0.17628008804589573, 1.1805945756091136e-5, 0.12448787278157, 0.001469

```

```

θ_multi = compartment(sol_multi, result_multi, :θ)
S_multi = compartment(sol_multi, result_multi, :S)
R_multi = compartment(sol_multi, result_multi, :R)
I_multi = compartment(sol_multi, result_multi, :I)

```

```

plot(sol_multi.t, S_multi, label="S", lw=2)
plot!(sol_multi.t, I_multi, label="I1 + I2", lw=2)
plot!(sol_multi.t, R_multi, label="R", lw=2)
xlabel!("Time")
ylabel!("Fraction")
title!("Two-stage infection (Catalyst-defined)")

```

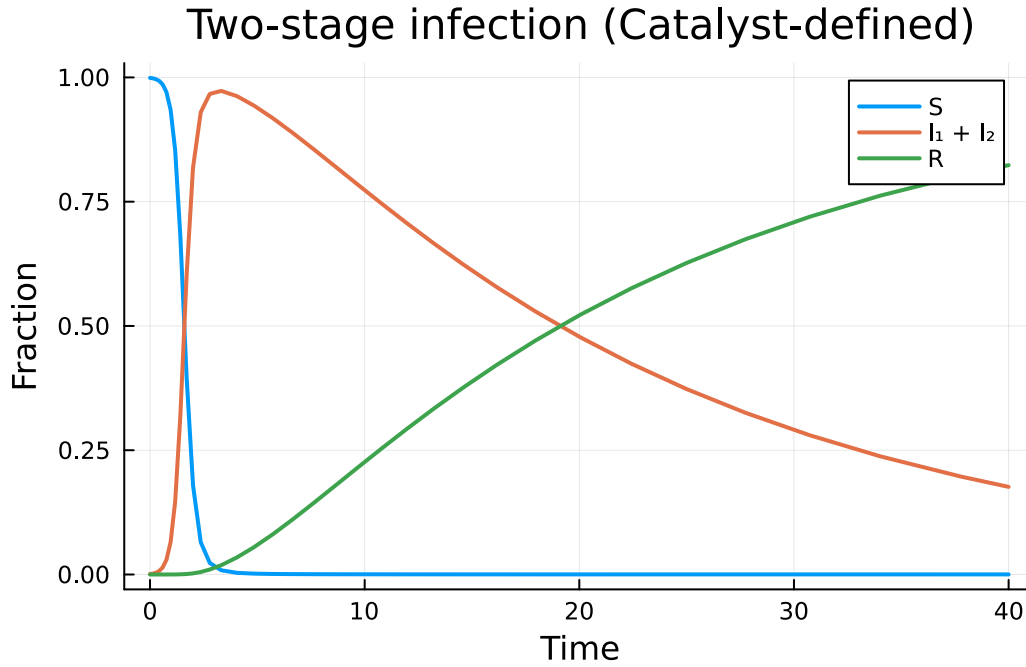


Figure 4: Two-stage infectious disease on a Poisson network. The acute phase (I_1) has higher transmissibility than the chronic phase (I_2).

Symbolic R_0

We can also compute the basic reproduction number symbolically for the multi-stage model:

```

R0_multi = basic_reproduction_number(scm_multi)
println("R0 = ", R0_multi)

```

$$R_0 = (((\beta_1 + \gamma_{1_cat}) * (\beta_2 + \gamma_{2_cat}) - \gamma_{1_cat} * \gamma_{2_cat}) * \kappa) / ((\beta_1 + \gamma_{1_cat}) * (\beta_2 + \gamma_{2_cat}))$$

For two infectious stages with transmission rates β_1, β_2 and progression rates γ_1, γ_2 , the transmissibility across an edge is:

$$T = 1 - \frac{\gamma_1}{\beta_1 + \gamma_1} \cdot \frac{\gamma_2}{\beta_2 + \gamma_2}$$

and $R_0 = T \cdot \psi''(1)/\psi'(1)$, where the excess degree ratio $\psi''(1)/\psi'(1) = \kappa$ for a Poisson distribution.

Simulation validation

We verify the ODE prediction against Gillespie SSA on Erdős–Rényi networks with mean degree $\kappa = 10$, using `NetworkOutbreaks.jl`.

```
include("../_validation.jl")
```

```
t_g, μ_g, σ_g = gillespie_ribbon(
    sir_model(), Dict{:β ⇒ β_val, :γ ⇒ γ_val},
    poisson_graph_builder(1000, κ_val);
    N = 1000, n_graphs = 5, nsims_per_graph = 20,
    tspan = tspan, seed_fraction = ε)
```

```
([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5 ... 35.5, 36.0, 36.5, 37.0, 37.5, 38.0, 38.5,
```

```
plot(t_g, μ_g[:I], ribbon = σ_g[:I], label = "SSA (mean ± 1σ)",
     color = :red, fillalpha = 0.2, linealpha = 0.6)
plot!(sol.t, I_vals, label = "EBCM Catalyst",
     color = :red, linewidth = 2)
xlabel!("Time"); ylabel!("Fraction infected")
title!("Catalyst-defined EBCM vs SSA")
```

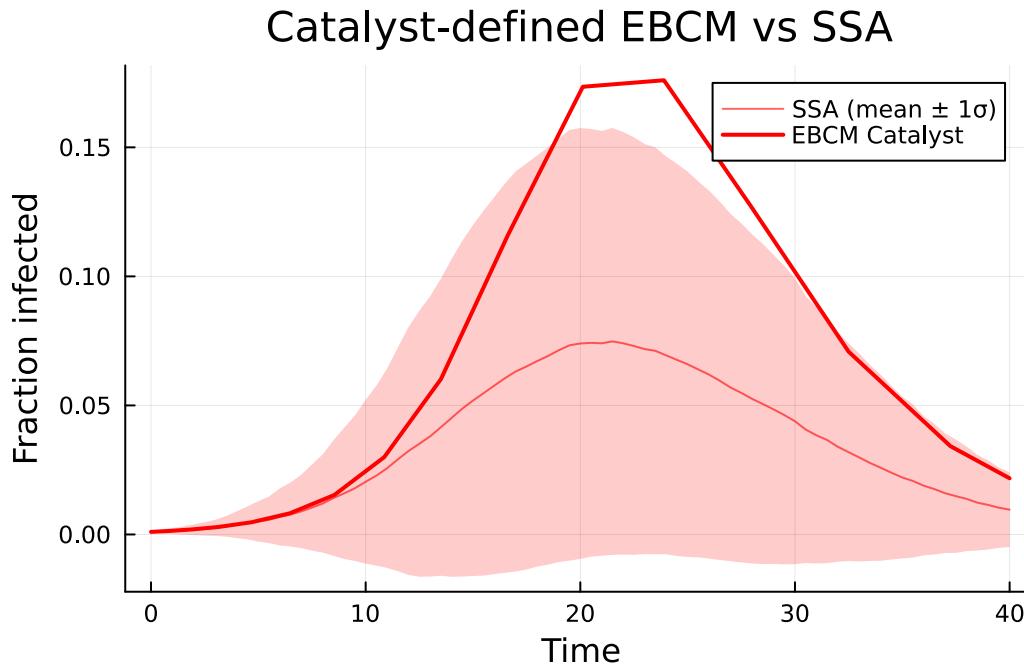


Figure 5: Catalyst-defined EBCM (red line) vs Gillespie SSA mean $\pm 1\sigma$ (red ribbon) on Poisson $\kappa=10$.

Benefits of the Catalyst adapter

The `progression_from_catalyst` function provides several advantages:

1. **Familiar syntax:** Users of Catalyst.jl can specify disease compartment transitions using the same `@reaction_network` macro they use for chemical kinetics.
2. **Validation:** Catalyst's reaction parsing catches malformed transitions (e.g., bimolecular reactions, which are not supported in the edge-based framework).
3. **Ecosystem integration:** Models defined in Catalyst can be analyzed with Catalyst's own tools (e.g., species graphs, conservation laws) before converting to edge-based form.
4. **Rapid prototyping:** Complex multi-stage progressions can be written in a few lines of Catalyst DSL, rather than manually constructing `DiseaseStage` and `DiseaseTransition` vectors.

The key constraint is that only unimolecular reactions are supported — each reaction must have exactly one substrate and one product, representing the progression of a single individual between disease compartments.